

Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 16, 2025
Activity 4

Files

<https://github.com/CGalloway3/CST-250-Projects/tree/master/Activity4>

Video

<https://www.loom.com/share/273e6b74002348859e360bd1eecfbf6d>

Part 1 Activity Submission

FLOW CHART

/*

* Chad Galloway

* CST - 250 Programming in C# II

* 11/16/2025

* Pizza Order System

* Activity 4

* References:

*/

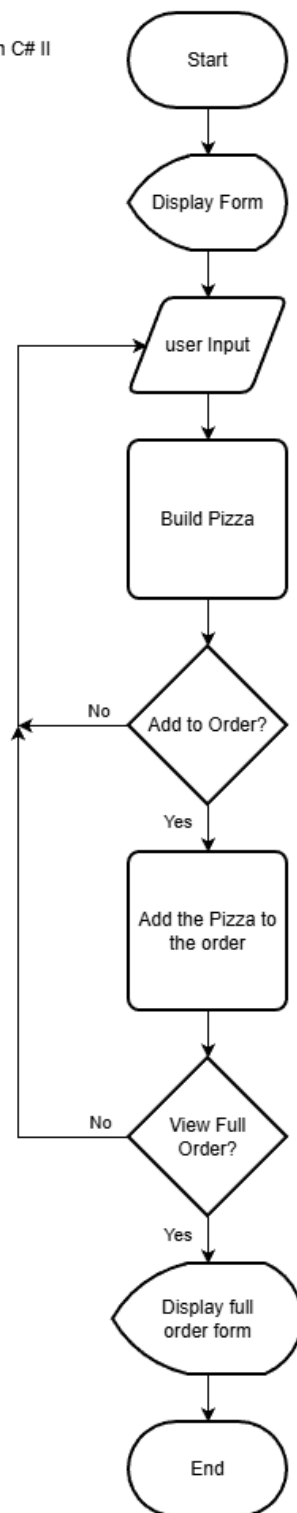


Figure 1: Flow chart of Activity 4

UML Class Diagram

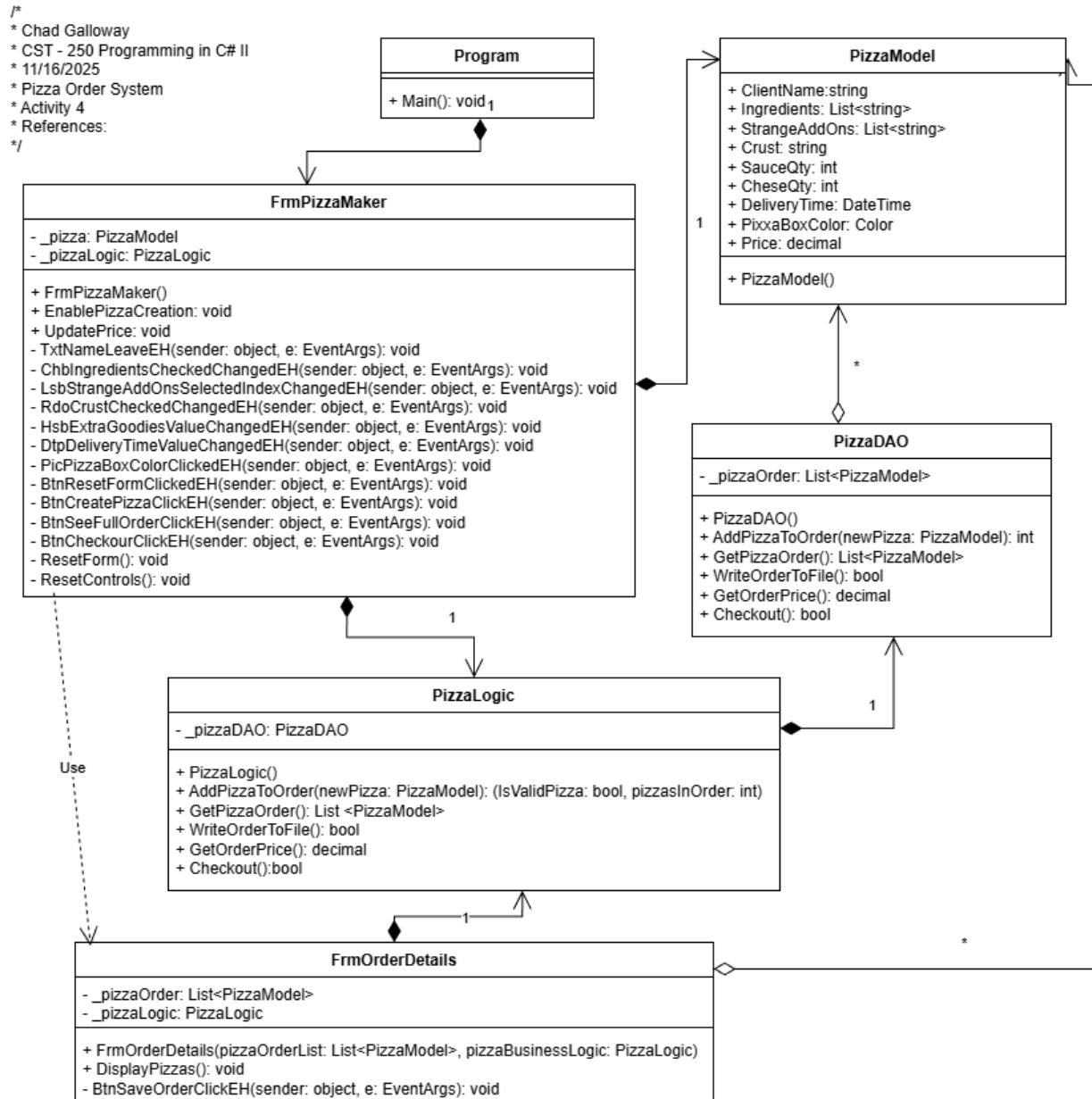


Figure 2: Final UML Class Diagram for the Activity 4 project.

Screen Shots Section 1

The screenshot shows a web application titled "Pizza Maker" in a browser window. The interface is organized into several sections:

- Name:** A text input field with a blue underline.
- Ingredients:** A list of eight items, each with an unchecked checkbox: Pepperoni, Pineapple, Bacon, Sausage, Olives, Peppers, Mushrooms, and Tomatoes.
- Delivery Time:** A date and time selector showing "11/12/25 12:00" with a calendar icon and a dropdown arrow.
- Pizza Box Color:** A large, empty rectangular box for color selection.
- Pizza Price:** A label "Pizza Price:" followed by "\$0" in red text.
- Buttons:** Two buttons at the bottom right: "Reset Form" and "Create Pizza".
- Strange Add Ons:** A vertical list of items: Hotdogs, Eggplant, Artichoke Hearts, Eggs, Peanut Butter, Prosciutto, Honey, and Chilli Thread.
- Crust:** A list of four options, each with an unchecked radio button: Thin Crust, Deep Dish, Stuffed Crust, and Gluten Free.
- Extra Goodies:** A section containing two sliders:
 - Amount of Sauce:** A slider with a value of "00".
 - Amount of Cheese:** A slider with a value of "00".

Figure 1-3: App Running for the first time after completing section one.

Screen Shots Section 2

The screenshot shows a web application titled "Pizza Maker" with a light gray background. The form is organized into several sections:

- Name:** A text input field containing "Steve Fisk".
- Delivery Time:** A date and time picker showing "11/12/2025 12:05".
- Ingredients:** A section with two columns of checkboxes:
 - Left column: ☐ Pepperoni, ☐ Bacon, ☐ Olives, ☐ Mushrooms.
 - Right column: ☐ Pineapple, ☒ Sausage, ☐ Peppers, ☐ Tomatoes.
- Pizza Box Color:** A large rectangular color picker showing a bright green color.
- Pizza Price:** A text label showing "\$16.00" in red.
- Buttons:** Two buttons at the bottom right: "Reset Form" and "Create Pizza".
- Strange Add Ons:** A vertical list of items with a scrollbar, where "Artichoke Hearts" is selected and highlighted in blue. The items are: Hotdogs, Eggplant, Artichoke Hearts, Eggs, Peanut Butter, Prosciutto, Honey, and Chilli Thread.
- Crust:** A section with four radio button options: ☐ Thin Crust, ☐ Deep Dish, ☐ Stuffed Crust, and ☐ Gluten Free.
- Extra Goodies:** A section with two sliders:
 - "Amount of Sauce" with a value of 59.
 - "Amount of Cheese" with a value of 62.

Figure 2-1: Screenshot of successful output after the addition of event handlers for the UI in section 2.

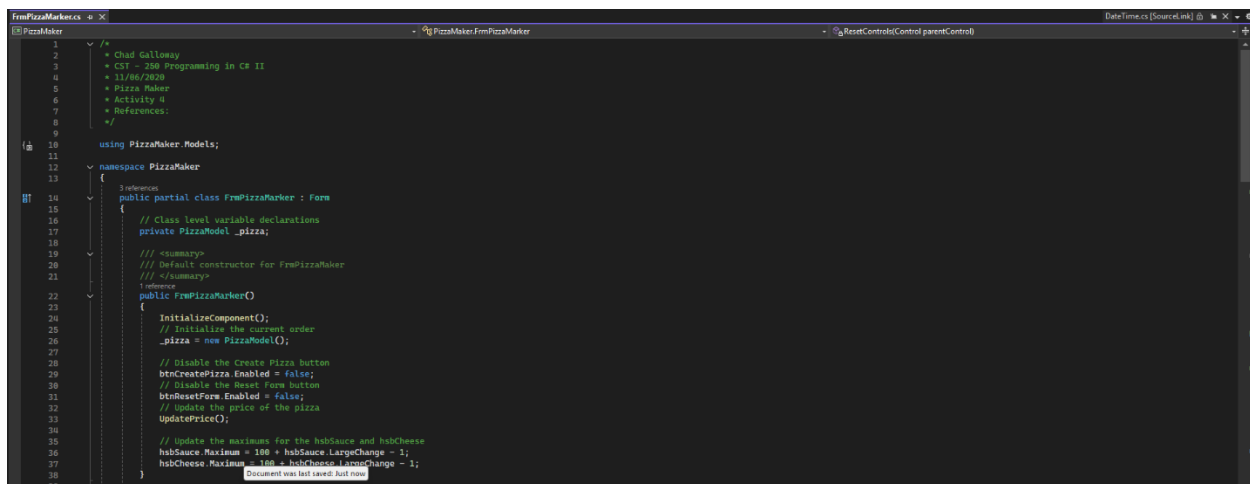


Figure 2-2: Screenshot of FrmPizzaMaker.cs citations and constructor

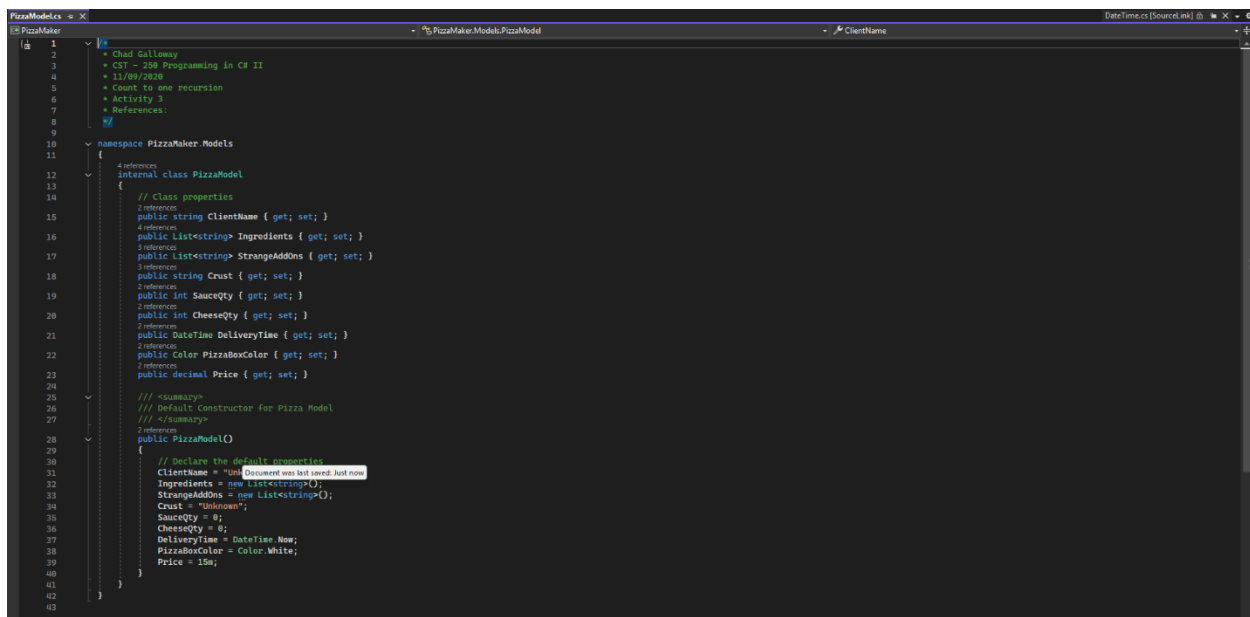


Figure 2-3: Screenshot of PizzaModel.cs citations and constructor

```

frmPizzaMaker.cs
// Summary
// Enable the reset and create buttons
// for the order pizza form
// Summary
// Summary
public void EnablePizzaCreation()
{
    // Enable the create pizza button
    btnCreatePizza.Enabled = true;
    // Enable the Reset Form button
    btnResetForm.Enabled = true;
}

// Summary
// Leave Event handler for txtName
// Summary
// param name="sender" type="object"
// param name="e" type="EventArgs"
private void TxtNameLeaveEH(object sender, EventArgs e)
{
    // Set the Pizza client name to test name
    _pizza.ClientName = txtName.Text;
    // Call enable pizza creation method
    EnablePizzaCreation();
}

// Summary
// Checked Changed event handler for all the ingredient checkboxes
// Summary
// param name="sender" type="object"
// param name="e" type="EventArgs"
private void ChbIngredientsCheckedChangedEH(object sender, EventArgs e)
{
    // Get the checkbox from the sender parameter
    CheckBox checkbox = sender as CheckBox;
    // Make sure the checkbox is not null
    if (checkbox != null)
    {
        // If the checkbox is checked, add the ingredient to the pizza
        if (checkbox.Checked)
        {
            // Add the current ingredient to the pizza
            _pizza.Ingredients.Add(checkbox.Text);
        }
        // If the checkbox is not checked, remove the ingredient
        else
        {
            // Remove the current ingredient from the pizza
            _pizza.Ingredients.Remove(checkbox.Text);
        }
    }
    // Update the price of the pizza
    UpdatePrice();
}

```

Figure 2-4: Screenshot of EnablePizzaCreation, TxtNameLeaveEH, and ChbIngredientsCheckedChangedEH methods.

```

frmPizzaMaker.cs
// Summary
// Selected Index Changed event handler for lstStrangeAddOns
// Summary
// param name="sender" type="object"
// param name="e" type="EventArgs"
private void LstStrangeAddOnsSelectedIndexChangedEH(object sender, EventArgs e)
{
    // Get the list of the selected ingredients and set the strange addons property of the pizza
    _pizza.StrangeAddOns = lstStrangeAddOns.SelectedItem as List<String>();
    // Update the price of the pizza
    UpdatePrice();
}

// Summary
// Update the price of the pizza
// Summary
// Summary
public void UpdatePrice()
{
    // Declare and initialize
    decimal price = 15;

    // Add 50 cents for each ingredient
    price += (_pizza.Ingredients.Count * .50m);

    // Add 50 cents for each special add on
    price += (_pizza.StrangeAddOns.Count * .50m);

    // Add 1 if the crust is gluten free
    if (_pizza.Crust == "Gluten Free")
    {
        price += 1;
    }

    // Update the price of the pizza
    _pizza.Price = price;
    // Update lblPriceAmount
    lblPriceAmount.Text = $"{price:C2}";
}

// Summary
// Checked Changed event handler for crust radio buttons
// Summary
// param name="sender" type="object"
// param name="e" type="EventArgs"
private void RdoCrustCheckedChangedEH(object sender, EventArgs e)
{
    // Get the radio button from the sender object
    RadioButton radioButton = sender as RadioButton;
    // Make sure this radio button is not null
    if (radioButton != null && radioButton.Checked)
    {
        _pizza.Crust = radioButton.Text;
    }
    // Update the price
    UpdatePrice();
}

```

Figure 2-5: Screenshot of LstStrangeAddOnsSelectedIndexChangedEH, UpdatePrice, and RdoCrustCheckedChangedEH methods.

```

154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215
// PizzaMaker
/// <summary>
/// Value changed event handler for the horizontal scroll bars
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
/// </summary>
private void HsbExtraGoodiesValueChangedEH(object sender, EventArgs e)
{
    // Cast the sender object to an HScrollBar
    HScrollBar scrollbar = sender as HScrollBar;
    // Make sure the bar is not null
    if (scrollbar != null)
    {
        // Check if the scroll bar is hsbSauce
        if (scrollbar == hsbSauce)
        {
            // Update the sauce quantity property using the scroll bars value
            _pizza.SauceQty = scrollbar.Value;
            // Update the label sauce value
            lblSauceAmount.Text = scrollbar.Value.ToString();
        }
        // Check if the scroll bar is hsbCheese
        else if (scrollbar == hsbCheese)
        {
            // Update the cheese quantity property using the scroll bars value
            _pizza.CheeseQty = scrollbar.Value;
            // Update the label cheese value
            lblCheeseAmount.Text = scrollbar.Value.ToString();
        }
    }
}

/// <summary>
/// Value changed event handler for the dtpDeliveryTime
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
/// </summary>
private void DtpDeliveryTimeValueChangedEH(object sender, EventArgs e)
{
    // Update the delivery time for the pizza
    _pizza.DeliveryTime = dtpDeliveryTime.Value;
}

/// <summary>
/// Click event handler for the picPizzaBoxColor
/// </summary>
/// <param name="sender"></param>
/// <param name="e"></param>
/// </summary>
private void PicPizzaBoxColorClickEH(object sender, EventArgs e)
{
    // Create a new color dialog object
    ColorDialog picPizzaBoxColorPicker = new ColorDialog();
    // Call the show dialog method
    DialogResult result = picPizzaBoxColorPicker.ShowDialog();
    // Check if the color picker returned ok
    if (result == DialogResult.OK)
    {
        // Set the Pizza box color
        _pizza.PizzaBoxColor = picPizzaBoxColorPicker.Color;
        // Set the color of the picture box
        picPizzaBoxColor.BackColor = picPizzaBoxColorPicker.Color;
    }
}

```

Figure 2-6: Screenshot of HsbExtraGoodiesValueChangedEH, DtpDeliveryTimeValueChangedEH, and PicPizzaBoxColorClickEH methods.

```

221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315
// PizzaMaker
/// <summary>
/// Reset the Pizza maker form
/// </summary>
/// </summary>
private void ResetForm()
{
    // set the pizza to a new instance
    _pizza = new PizzaModel();
    // Reset the controls on the new form
    ResetControls(this);
    // Update the price of the Pizza
    UpdatePrice();
}

/// <summary>
/// Reset the controls within the parent control
/// </summary>
/// <param name="parentControl"></param>
/// </summary>
private void ResetControls(Control parentControl)
{
    // Loop through the controls within the parent control
    foreach (Control control in parentControl.Controls)
    {
        // Get the type of the control
        Type controlType = control.GetType();
        // Save the type of the control as a string
        string type = controlType.Name.ToString();

        // Use a switch case to handle the resets
        switch (type)
        {
            case "TextBox":
                break;

            case "CheckBox":
                // Cast the control to a checkbox
                CheckBox checkbox = (CheckBox)control;
                // Make sure the checkbox is not checked
                checkbox.Checked = false;
                break;

            case "ListBox":
                break;

            case "RadioButton":
                // Cast the control to a radio button
                RadioButton radioButton = (RadioButton)control;
                // Make sure the radio button is not checked
                radioButton.Checked = false;
                break;

            case "HScrollBar":
                // Cast the control to a horizontal scroll bar
                HScrollBar hScrollBar = (HScrollBar)control;
                // Set the scroll bars value to 0
                hScrollBar.Value = 0;
                break;

            case "DateTimePicker":
                break;

            case "PictureBox":
                break;
        }

        // Check if the control has controls (children)
        if (control.HasChildren)
        {
            // Recursively call the Reset method using the current control
            ResetControls(control);
        }
    }
}
// End of ResetForm method

```

Figure 2-7: Screenshot of ResetForm and ResetControls methods. (Note: some of the cases are collapsed in the IDE to allow it all to fit in one screenshot)

```

1 2
3 3
4 4
5 5
6 6
7 7
8 8
9 9
10 10
11 11
12 12
13 13
14 14
15 15
16 16
17 17
18 18
19 19
20 20
21 21
22 22
23 23
24 24
25 25
26 26
27 27
28 28
29 29
30 30
31 31
32 32
33 33
34 34
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100
101 101
102 102
103 103
104 104
105 105
106 106
107 107
108 108
109 109
110 110
111 111
112 112
113 113
114 114
115 115
116 116
117 117
118 118
119 119
120 120
121 121
122 122
123 123
124 124
125 125
126 126
127 127
128 128
129 129
130 130
131 131
132 132
133 133
134 134
135 135
136 136
137 137
138 138
139 139
140 140
141 141
142 142
143 143
144 144
145 145
146 146
147 147
148 148
149 149
150 150
151 151
152 152
153 153
154 154
155 155
156 156
157 157
158 158
159 159
160 160
161 161
162 162
163 163
164 164
165 165
166 166
167 167
168 168
169 169
170 170
171 171
172 172
173 173
174 174
175 175
176 176
177 177
178 178
179 179
180 180
181 181
182 182
183 183
184 184
185 185
186 186
187 187
188 188
189 189
190 190
191 191
192 192
193 193
194 194
195 195
196 196
197 197
198 198
199 199
200 200
201 201
202 202
203 203
204 204
205 205
206 206
207 207
208 208
209 209
210 210
211 211
212 212
213 213
214 214
215 215
216 216
217 217
218 218
219 219
220 220
221 221
222 222
223 223
224 224
225 225
226 226
227 227
228 228
229 229
230 230
231 231
232 232
233 233
234 234
235 235
236 236
237 237
238 238
239 239
240 240
241 241
242 242
243 243
244 244
245 245
246 246
247 247
248 248
249 249
250 250
251 251
252 252
253 253
254 254
255 255
256 256
257 257
258 258
259 259
260 260
261 261
262 262
263 263
264 264
265 265
266 266
267 267
268 268
269 269
270 270
271 271
272 272
273 273
274 274
275 275
276 276
277 277
278 278
279 279
280 280
281 281
282 282
283 283
284 284
285 285
286 286
287 287
288 288
289 289
290 290
291 291
292 292
293 293
294 294
295 295
296 296
297 297
298 298
299 299
300 300
301 301
302 302
303 303
304 304
305 305
306 306
307 307
308 308
309 309
310 310
311 311
312 312
313 313
314 314
315 315
316 316
317 317
318 318
319 319
320 320
321 321
322 322
323 323
324 324
325 325
326 326
327 327
328 328
329 329
330 330
331 331
332 332
333 333
334 334
335 335
336 336
337 337
338 338
339 339
340 340
341 341
342 342
343 343
344 344
345 345
346 346
347 347
348 348
349 349
350 350
351 351
352 352
353 353
354 354
355 355
356 356
357 357
358 358
359 359
360 360
361 361
362 362
363 363
364 364
365 365
366 366
367 367
368 368
369 369
370 370
371 371
372 372
373 373
374 374
375 375
376 376
377 377
378 378
379 379
380 380
381 381
382 382
383 383
384 384
385 385
386 386
387 387
388 388
389 389
390 390
391 391
392 392
393 393
394 394
395 395
396 396
397 397
398 398
399 399
400 400
401 401
402 402
403 403
404 404
405 405
406 406
407 407
408 408
409 409
410 410
411 411
412 412
413 413
414 414
415 415
416 416
417 417
418 418
419 419
420 420
421 421
422 422
423 423
424 424
425 425
426 426
427 427
428 428
429 429
430 430
431 431
432 432
433 433
434 434
435 435
436 436
437 437
438 438
439 439
440 440
441 441
442 442
443 443
444 444
445 445
446 446
447 447
448 448
449 449
450 450
451 451
452 452
453 453
454 454
455 455
456 456
457 457
458 458
459 459
460 460
461 461
462 462
463 463
464 464
465 465
466 466
467 467
468 468
469 469
470 470
471 471
472 472
473 473
474 474
475 475
476 476
477 477
478 478
479 479
480 480
481 481
482 482
483 483
484 484
485 485
486 486
487 487
488 488
489 489
490 490
491 491
492 492
493 493
494 494
495 495
496 496
497 497
498 498
499 499
500 500
501 501
502 502
503 503
504 504
505 505
506 506
507 507
508 508
509 509
510 510
511 511
512 512
513 513
514 514
515 515
516 516
517 517
518 518
519 519
520 520
521 521
522 522
523 523
524 524
525 525
526 526
527 527
528 528
529 529
530 530
531 531
532 532
533 533
534 534
535 535
536 536
537 537
538 538
539 539
540 540
541 541
542 542
543 543
544 544
545 545
546 546
547 547
548 548
549 549
550 550
551 551
552 552
553 553
554 554
555 555
556 556
557 557
558 558
559 559
560 560
561 561
562 562
563 563
564 564
565 565
566 566
567 567
568 568
569 569
570 570
571 571
572 572
573 573
574 574
575 575
576 576
577 577
578 578
579 579
580 580
581 581
582 582
583 583
584 584
585 585
586 586
587 587
588 588
589 589
590 590
591 591
592 592
593 593
594 594
595 595
596 596
597 597
598 598
599 599
600 600
601 601
602 602
603 603
604 604
605 605
606 606
607 607
608 608
609 609
610 610
611 611
612 612
613 613
614 614
615 615
616 616
617 617
618 618
619 619
620 620
621 621
622 622
623 623
624 624
625 625
626 626
627 627
628 628
629 629
630 630
631 631
632 632
633 633
634 634
635 635
636 636
637 637
638 638
639 639
640 640
641 641
642 642
643 643
644 644
645 645
646 646
647 647
648 648
649 649
650 650
651 651
652 652
653 653
654 654
655 655
656 656
657 657
658 658
659 659
660 660
661 661
662 662
663 663
664 664
665 665
666 666
667 667
668 668
669 669
670 670
671 671
672 672
673 673
674 674
675 675
676 676
677 677
678 678
679 679
680 680
681 681
682 682
683 683
684 684
685 685
686 686
687 687
688 688
689 689
690 690
691 691
692 692
693 693
694 694
695 695
696 696
697 697
698 698
699 699
700 700
701 701
702 702
703 703
704 704
705 705
706 706
707 707
708 708
709 709
710 710
711 711
712 712
713 713
714 714
715 715
716 716
717 717
718 718
719 719
720 720
721 721
722 722
723 723
724 724
725 725
726 726
727 727
728 728
729 729
730 730
731 731
732 732
733 733
734 734
735 735
736 736
737 737
738 738
739 739
740 740
741 741
742 742
743 743
744 744
745 745
746 746
747 747
748 748
749 749
750 750
751 751
752 752
753 753
754 754
755 755
756 756
757 757
758 758
759 759
760 760
761 761
762 762
763 763
764 764
765 765
766 766
767 767
768 768
769 769
770 770
771 771
772 772
773 773
774 774
775 775
776 776
777 777
778 778
779 779
780 780
781 781
782 782
783 783
784 784
785 785
786 786
787 787
788 788
789 789
790 790
791 791
792 792
793 793
794 794
795 795
796 796
797 797
798 798
799 799
800 800
801 801
802 802
803 803
804 804
805 805
806 806
807 807
808 808
809 809
810 810
811 811
812 812
813 813
814 814
815 815
816 816
817 817
818 818
819 819
820 820
821 821
822 822
823 823
824 824
825 825
826 826
827 827
828 828
829 829
830 830
831 831
832 832
833 833
834 834
835 835
836 836
837 837
838 838
839 839
840 840
841 841
842 842
843 843
844 844
845 845
846 846
847 847
848 848
849 849
850 850
851 851
852 852
853 853
854 854
855 855
856 856
857 857
858 858
859 859
860 860
861 861
862 862
863 863
864 864
865 865
866 866
867 867
868 868
869 869
870 870
871 871
872 872
873 873
874 874
875 875
876 876
877 877
878 878
879 879
880 880
881 881
882 882
883 883
884 884
885 885
886 886
887 887
888 888
889 889
890 890
891 891
892 892
893 893
894 894
895 895
896 896
897 897
898 898
899 899
900 900
901 901
902 902
903 903
904 904
905 905
906 906
907 907
908 908
909 909
910 910
911 911
912 912
913 913
914 914
915 915
916 916
917 917
918 918
919 919
920 920
921 921
922 922
923 923
924 924
925 925
926 926
927 927
928 928
929 929
930 930
931 931
932 932
933 933
934 934
935 935
936 936
937 937
938 938
939 939
940 940
941 941
942 942
943 943
944 944
945 945
946 946
947 947
948 948
949 949
950 950
951 951
952 952
953 953
954 954
955 955
956 956
957 957
958 958
959 959
960 960
961 961
962 962
963 963
964 964
965 965
966 966
967 967
968 968
969 969
970 970
971 971
972 972
973 973
974 974
975 975
976 976
977 977
978 978
979 979
980 980
981 981
982 982
983 983
984 984
985 985
986 986
987 987
988 988
989 989
990 990
991 991
992 992
993 993
994 994
995 995
996 996
997 997
998 998
999 999
1000 1000

```

Figure 2-8: Screenshot of the complete FrmPizzaMaker.cs showing the partial class organized with every method and summary collapsed.

UML Class Diagram

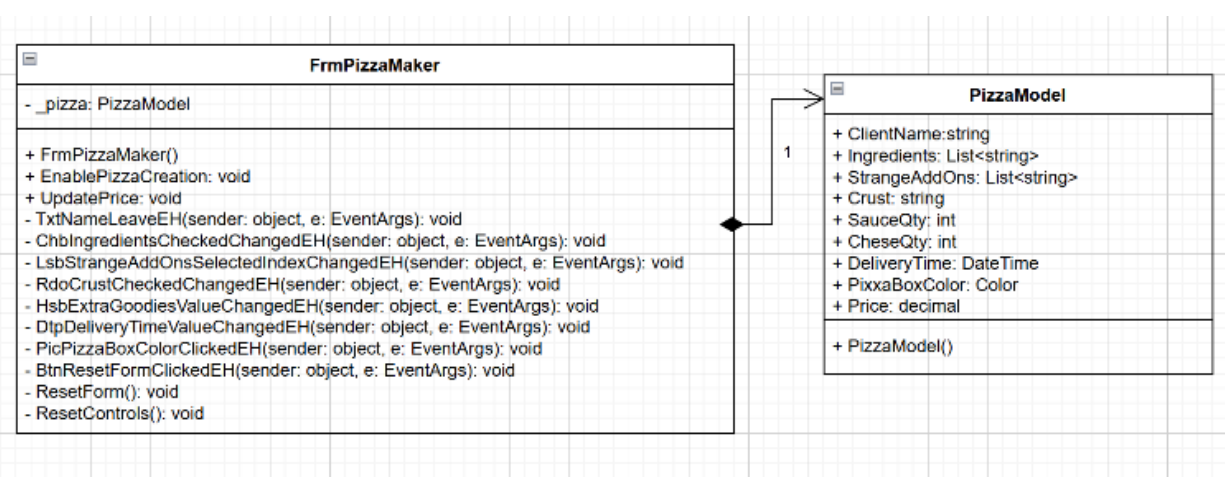


Figure 2-9: UML Class Diagram of the Pizza Maker form and the PizzaModel class.

Screen Shots Section 3

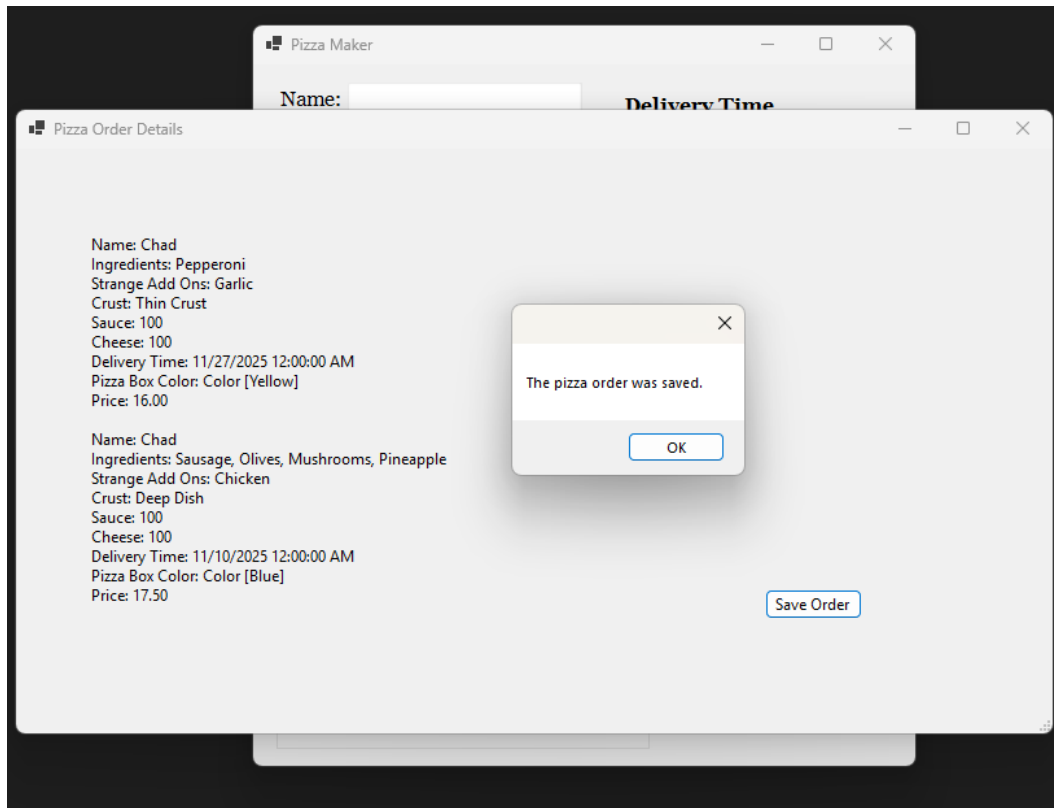


Figure 3-1: Part 3 App Running with save order confirmation message box on screen as per section 3 instructions.

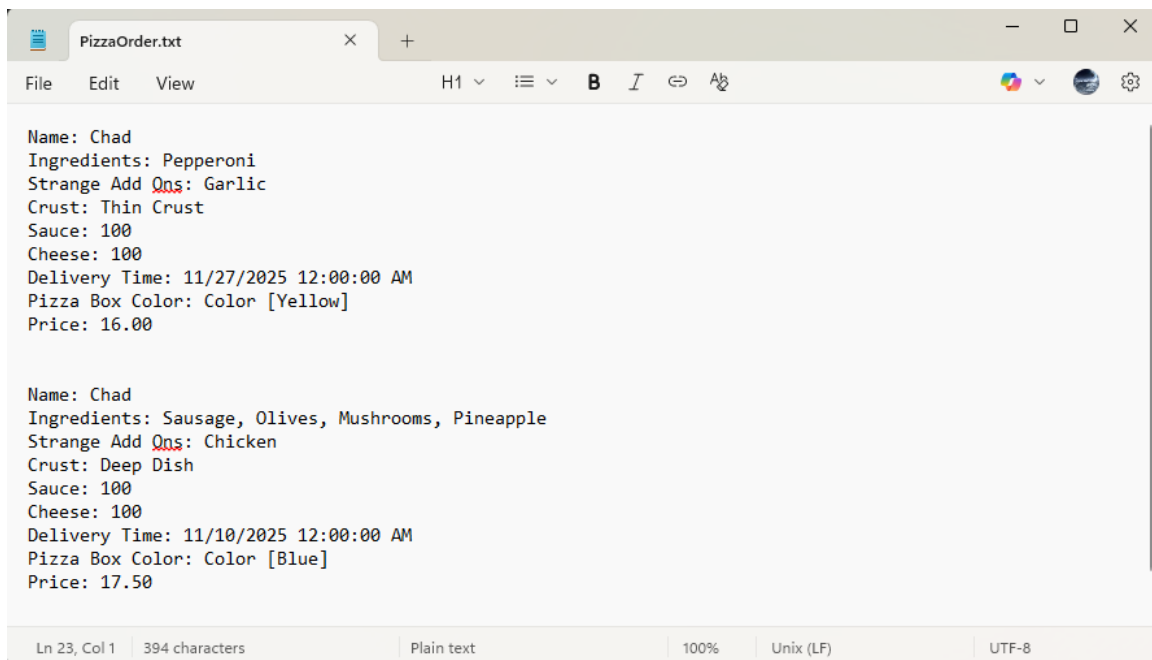


Figure 3-2: Screenshot of successful output to the file for the above screen shot in figure 3-1

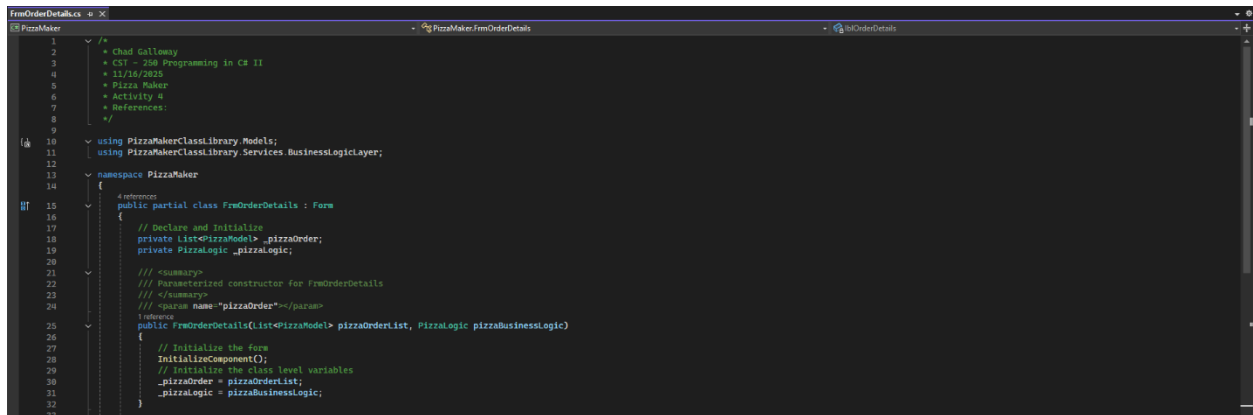


Figure 3-3: Screenshot of citations and constructor for the FrmOrderDetails

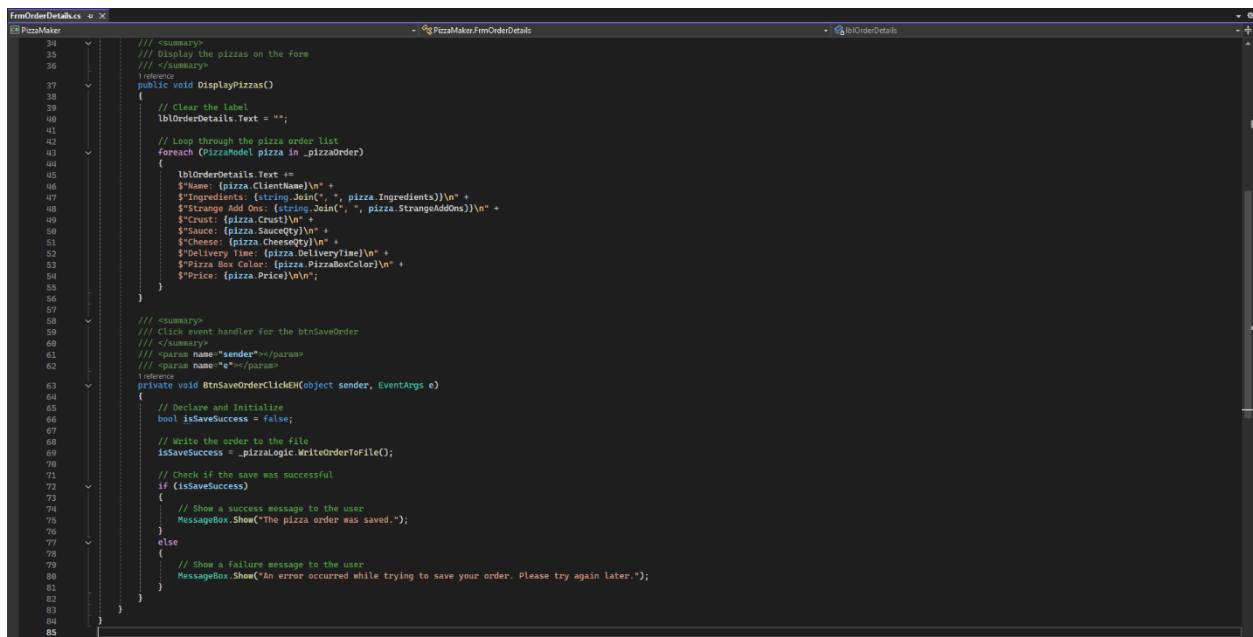


Figure 3-4: Screenshot of DisplayPizzas and BtnSaveOrderClickEH in the FrmOrderDetails

```

236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283
// 
// Click event handler for the create pizza button
// 
// 
// 
// void
private void BtnCreatePizzaClickEH(object sender, EventArgs e)
{
    // Declare and Initialize
    bool isValidPizza = false;
    int pizzasInOrder = -1;

    // Use the pizza logic to call add pizza to order
    (isValidPizza, pizzasInOrder) = _pizzaLogic.AddPizzaToOrder(_pizza);

    // Check if the pizza was valid
    if (isValidPizza)
    {
        // Enable the see full order button
        btnSeeFullOrder.Enabled = true;
        // Reset the form
        ResetForm();
    }
    else
    {
        // Display a failure message to the user
        MessageBox.Show("Your pizza order is not complete.");
    }
}

// 
// Click event handler for btnSeeFullOrder
// 
// 
// 
// void
private void BtnSeeFullOrderClickEH(object sender, EventArgs e)
{
    // Declare and Initialize
    List<PizzaModel> pizzaList;
    // Get the pizza list from pizzaLogic
    pizzaList = _pizzaLogic.GetPizzaOrder();
    // Create a new form with the pizza list
    FrmOrderDetails frmOrderDetails = new FrmOrderDetails(pizzaList, _pizzaLogic);
    // Update the label with the pizza order
    frmOrderDetails.DisplayPizzas();
    // Show the form
    frmOrderDetails.ShowDialog();
}

```

Figure 3-5: Screenshot of the BtnCreatePizzaClickEH, and BtnSeeFullOrderClickEH methods in FrmPizzaMaker.

```

29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
// 
// Add a new pizza to the current order
// 
// 
// bool, int
public (bool isValidPizza, int pizzasInOrder) AddPizzaToOrder(PizzaModel newPizza)
{
    // Declare and Initialize
    int pizzas = -1;

    // Call the DAO AddPizzaToOrder
    pizzas = _pizzaDAO.AddPizzaToOrder(newPizza);
    // Return the pizzas variable
    return (true, pizzas);
}

// 
// Get the list of pizzas in the current order
// 
// List<PizzaModel>
public List<PizzaModel> GetPizzaOrder()
{
    // Get and return the GetPizzaOrder from the DAO
    return _pizzaDAO.GetPizzaOrder();
}

// 
// Write the pizza order to a text file
// 
// bool
public bool WriteOrderToFile()
{
    // Get and return write order to file from the DAO
    return _pizzaDAO.WriteOrderToFile();
}

```

Figure 3-6: Screenshot of the AddPizzaToOrder, GetPizzaOrder, and WriteOrderToFile methods in the PizzaLogic

```

28 // <summary>
29 // Add a pizza to the current order
30 // </summary>
31 // <param name="newPizza"></param>
32 // <returns></returns>
33 // reference
34 public int AddPizzaToOrder(PizzaModel newPizza)
35 {
36     // Add the new pizza to the _pizzaOrder list
37     _pizzaOrder.Add (newPizza);
38     // Return the number of pizzas in _pizzaOrder
39     return _pizzaOrder.Count;
40 }
41
42 // <summary> Get the list of pizzas in the current order
43 // reference
44 public List<PizzaModel> GetPizzaOrder()
45 {
46     // Return the pizzaOrder list
47     return _pizzaOrder;
48 }
49
50 // <summary> Write the pizza order to a text file
51 // reference
52 public bool WriteOrderToFile()
53 {
54     // Declare and initialize
55     string filePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "App_Data");
56     string pizzaString = "";
57
58     // Check if the directory exists
59     if (!Directory.Exists(filePath))
60     {
61         // Create the directory
62         Directory.CreateDirectory(filePath);
63     }
64
65     // Set up a try-catch for the file writer
66     try
67     {
68         // Create a using statement for StreamWriter
69         using (StreamWriter streamWriter = new StreamWriter(Path.Combine(filePath, "PizzaOrder.txt")))
70         {
71             // Loop through the pizza order list
72             foreach (PizzaModel pizza in _pizzaOrder)
73             {
74                 pizzaString =
75                 $"Name: {pizza.ClientName}\n" +
76                 $"Ingredients: {string.Join(", ", pizza.Ingredients)}\n" +
77                 $"Strange Add Ons: {string.Join(", ", pizza.StrangeAddOns)}\n" +
78                 $"Crust: {pizza.Crust}\n" +
79                 $"Sauce: {pizza.SauceQty}\n" +
80                 $"Cheese: {pizza.CheeseQty}\n" +
81                 $"Delivery time: {pizza.DeliveryTime}\n" +
82                 $"Pizza Box Color: {pizza.PizzaBoxColor}\n" +
83                 $"Price: {pizza.Price}\n";
84                 streamWriter.WriteLine(pizzaString);
85             }
86
87             // Return true
88             return true;
89         }
90     }
91     catch
92     {
93         // Return false
94         return false;
95     }
96 }
97
98 }
99
100
101

```

Figure 3-7: Screenshot of the AddPizzaToOrder, GetPizzaOrder, and WriteOrderToFile methods in the PizzaDAO

UML Class Diagram

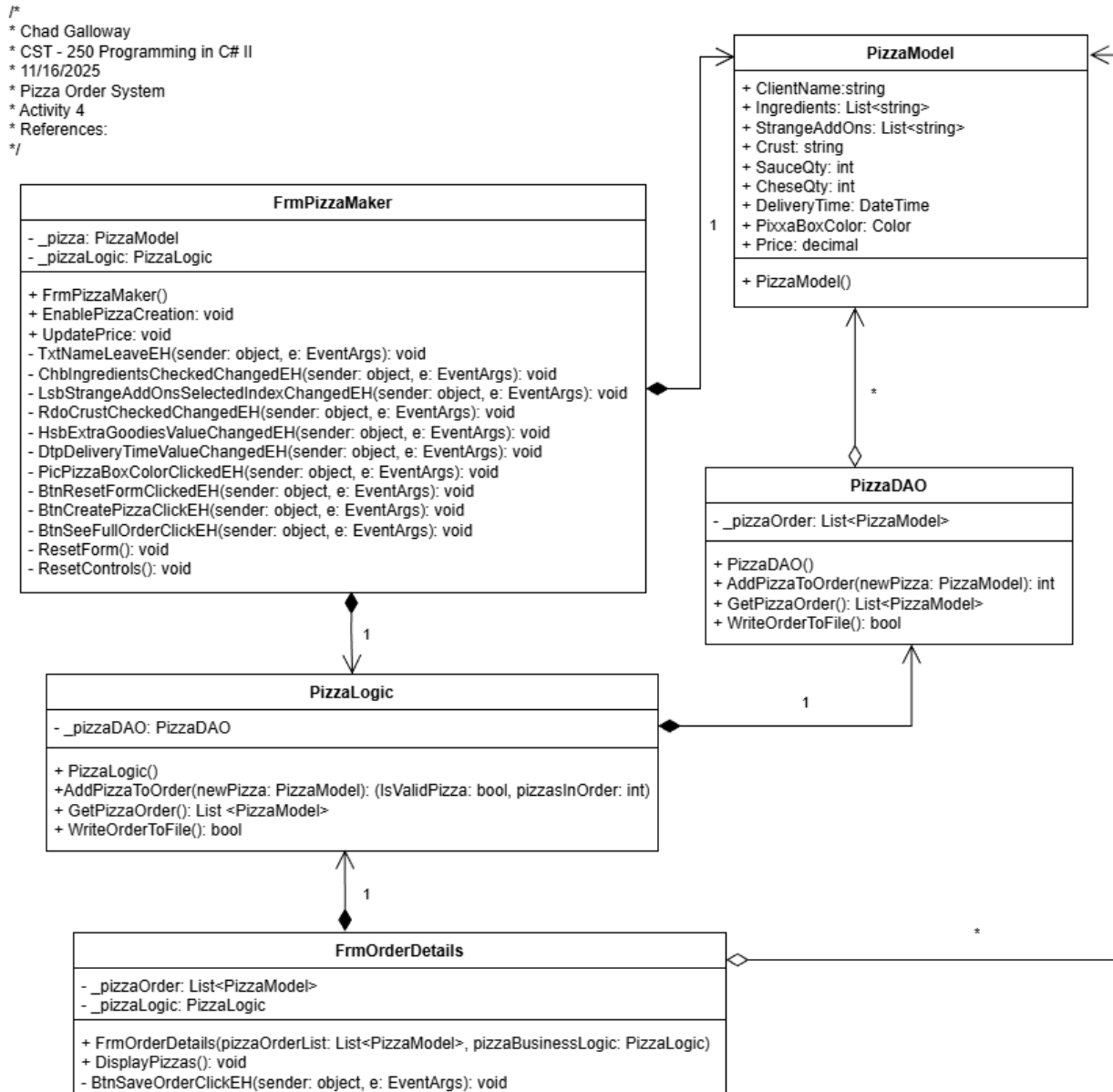


Figure 3-8: UML Class Diagram for the PizzaMaker project as it stands before challenges have been implemented.

Part 2 Challenges

Screen Shots

The screenshot shows a web application titled "Pizza Maker". It features several input fields and checkboxes for customizing a pizza. A modal dialog box is displayed in the center, indicating that the order is not complete due to missing information.

Name:

Delivery Time
11/10/2025 12:00

Ingredients

☐ Pepperoni	☐ Pineapple
☒ Bacon	☐ Sausage
☐ Olives	☐ Peppers
☐ Mushrooms	☐ Tomatoes

Pizza Box Color

Pizza Price: \$16.00

Strange Add On

- Hotdogs
- Eggplant
- Artichoke Hearts
- Eggs
- Peanut Butter
- Prosciutto
- Honey
- Chilli Thread

Extra Goodies

Amount of Sauce 20

Amount of Cheese 00

Gluten Free ☐

Pizzas on order

Total Order Price

Checkout

Create Pizza

See Full Order

Info

on last pizza

Your pizza order is not complete.

OK

Figure C-1: Screenshot of the Pizza validation challenge. The order fails because there is no name and there is no cheese.

Pizza Maker

Name:

Ingredients

☐ Pepperoni

☐ Pineapple

☐ Bacon

☐ Sausage

☐ Olives

☐ Peppers

☐ Mushrooms

☐ Tomatoes

Strange Add Ons

Hotdogs
Eggplant
Artichoke Hearts
Eggs
Peanut Butter
Prosciutto
Honey
Chilli Thread

Crust

☐ Thin Crust

☐ Deep Dish

☐ Stuffed Crust

☐ Gluten Free

Extra Goodies

Amount of Sauce 0

Amount of Cheese 0

Delivery Time

11/10/2025 12:00

Pizza Box Color

Pizza Price: \$15.00

Reset Form

Create Pizza

See Full Order

Order Info

Name on last pizza

Chad

Pizzas on order

1

Total Order Price

\$16.00

Checkout

Figure C-2: Screenshot of the app after fixing the missing data in figure C-1 and successfully adding a pizza to the order. The order info values get populated in that group box.

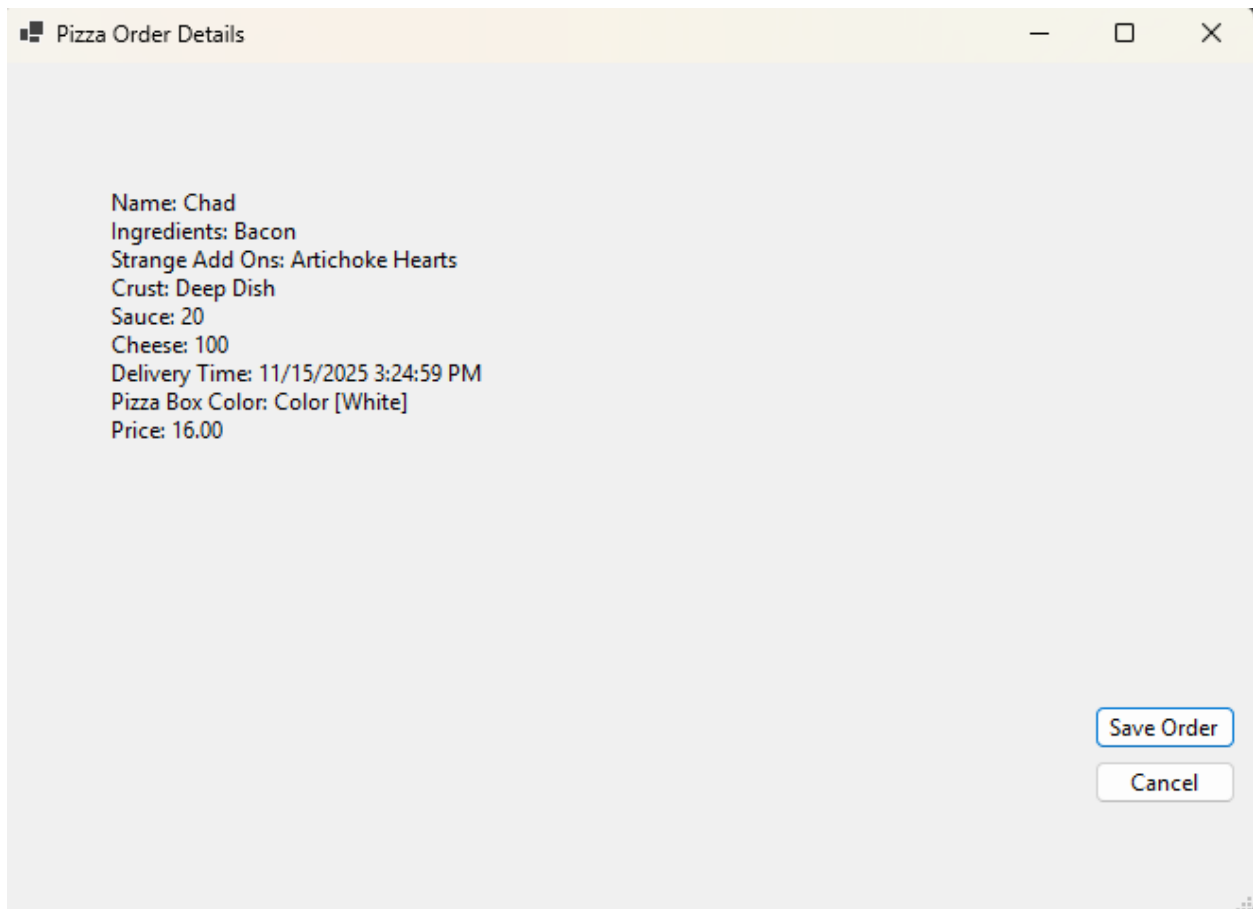


Figure C-3: Screenshot of the order when the see full order button in Figure C-2 is pressed.

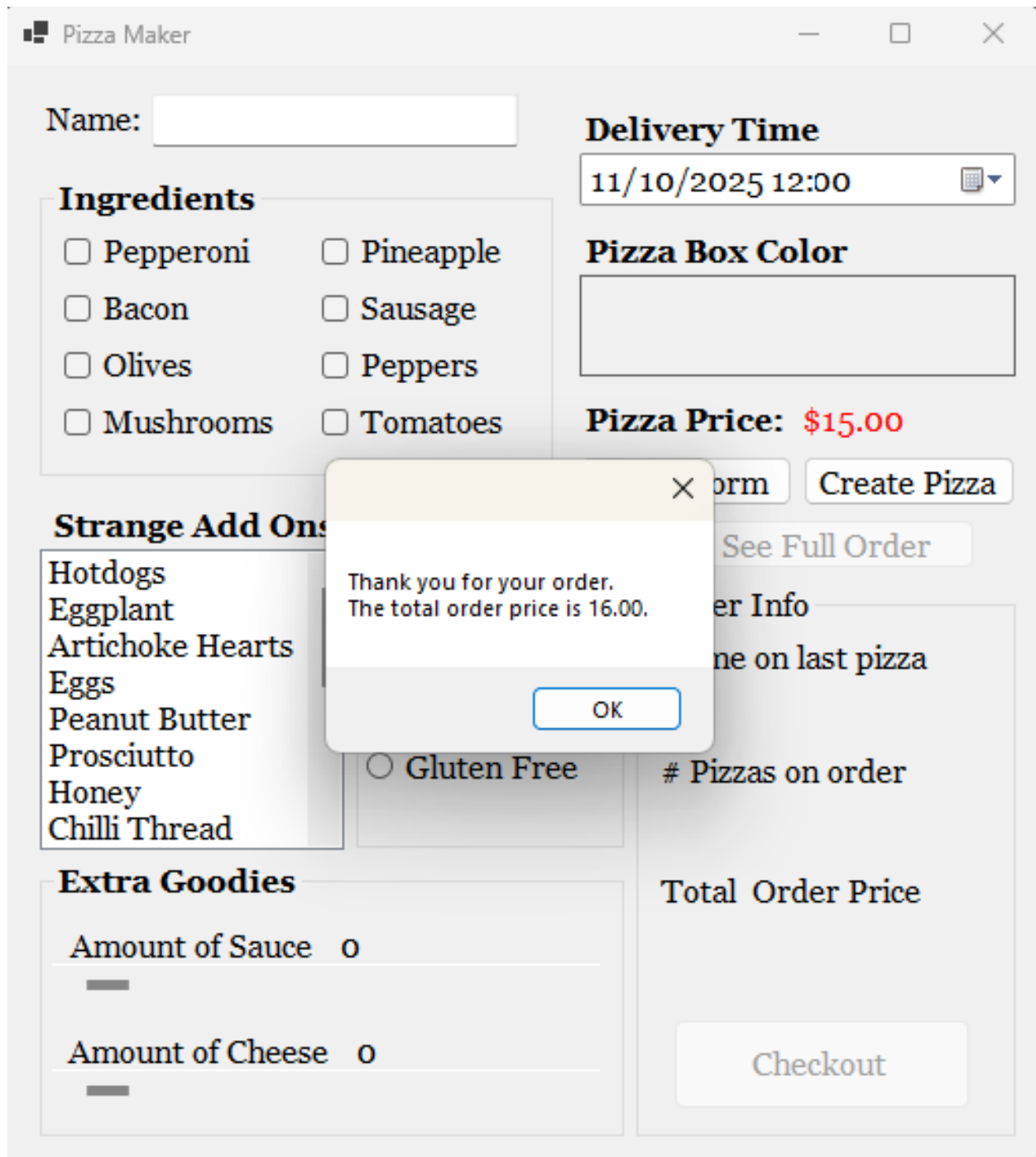


Figure C-4: Screenshot of the app after canceling the full order window by pressing the cancel button shown in Figure C-3 and then pressing the checkout button. The app is fully operational and robust and can easily pass over all the order data to the checkout backend in the future instead of simply displaying a message box.

```

153  /// <summary>
154  /// Calculate and return the order price
155  /// </summary>
156  /// <returns></returns>
157  public decimal GetOrderPrice()
158  {
159      // Declare and initialize
160      decimal priceTotal = 0;
161
162      // Add the price of each pizza to the total
163      foreach (PizzaModel pizza in _pizzaOrder)
164      {
165          priceTotal += pizza.Price;
166      }
167
168      // return the result
169      return priceTotal;
170  }
171
172  /// <summary>
173  /// Checkout method for the data object
174  /// </summary>
175  /// <returns></returns>
176  public bool Checkout()
177  {
178      // Enter logic for checking out at a later date
179      _pizzaOrder.Clear();
180      return true;
181  }
182  }
183

```

Figure C-5: Screenshot of the PizzaDAO GetOrderPrice and Checkout methods.

```

81  /// <summary>
82  /// Get the order price
83  /// </summary>
84  /// <returns></returns>
85  public decimal GetOrderPrice()
86  {
87      return _pizzaDAO.GetOrderPrice();
88  }
89
90  /// <summary>
91  /// Checkout logic for orders
92  /// </summary>
93  /// <returns></returns>
94  public bool Checkout()
95  {
96      return _pizzaDAO.Checkout();
97  }
98
99
100
101

```

Figure C-6: Screenshot of the PizzaLogic GetOrderPrice and Checkout methods.

```

289  /// <summary>
290  /// Click event handler for the btnCheckout
291  /// </summary>
292  /// <param name="sender"></param>
293  /// <param name="e"></param>
294  private void BtnCheckoutClickEH(object sender, EventArgs e)
295  {
296      // Declare and initialize
297      decimal orderPrice = _pizzaLogic.GetOrderPrice();
298
299      // Try to check out
300      if (_pizzaLogic.Checkout())
301      {
302          btnSeeFullOrder.Enabled = false;
303          btnCheckout.Enabled = false;
304          lblNumberOfPizzasAmount.Text = "";
305          lblTotalOrderPriceAmount.Text = "";
306          lblOrderNameAmount.Text = "";
307
308          MessageBox.Show($"Thank you for your order.\nThe total order price is {orderPrice}.");
309      }
310      else
311      {
312          MessageBox.Show("Checkout failed");
313      }
314  }
315

```

Figure C-7: Screenshot of the Pizza Maker Form's checkout button event handler.

```

86
87  /// <summary>
88  /// Event handler for the cancel button
89  /// </summary>
90  /// <param name="sender"></param>
91  /// <param name="e"></param>
92  private void BtnCancelClickEH(object sender, EventArgs e)
93  {
94      this.Close();
95  }
96
97
98

```

Figure C-8: Screenshot of the Order Details' cancel button event handler.

Part 3 Topic 5 Planning

FLOW CHART

```
/*  
* Chad Galloway  
* CST - 250 Programming in C# II  
* 11/16/2025  
* Wack-a-Mole Game  
* Activity 4 part 3 week 5 prep  
* References:  
*/
```

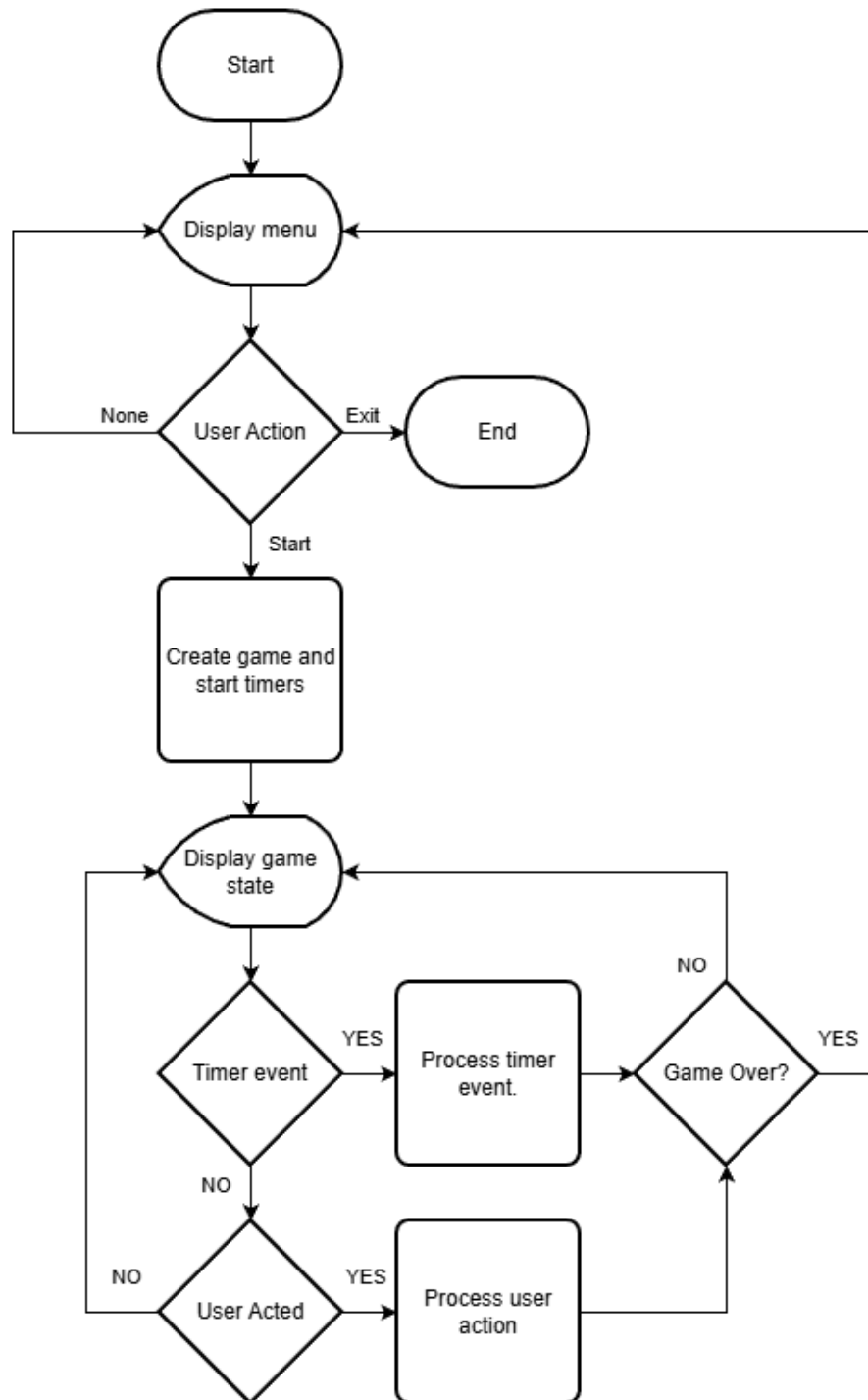


Figure 4-1: Preliminary flow chart of week five whack-a-mole game.

UML Class Diagram

```
/*  
* Chad Galloway  
* CST - 250 Programming in C# II  
* 11/16/2025  
* Wack-a-Mole Game  
* Activity 4 part 3 week 5 prep  
* References:  
*/
```

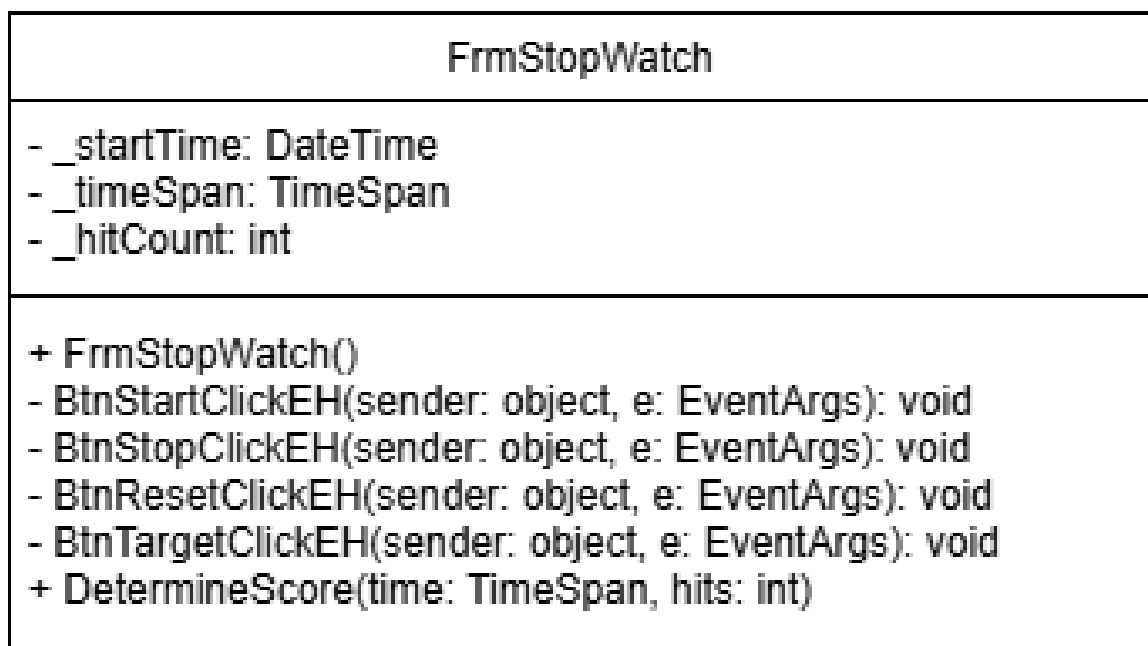


Figure 4-2: Preliminary UML Class Diagram for the week 5 whack-a-mole project